claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a1114c69169837ee4.jsonl`
- SHA-256: `45c29155bc673d318d6375d30eb976fa7c9d8e38b0b9f754b7b503337382e896`
- Source modified: `2026-06-22T15:04:37+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T15:03:09.420Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: robustness-security
Severity: major
Title: OverflowError on shot_count='inf'/'1e400' aborts the entire ingest mid-loop
File: backend/main.py:462-465
Detail: In _ingest_remote_segments, `int(float(sc))` is wrapped only in `except ValueError`. A shot_count value of `inf`, `-inf`, or `1e400` makes `float()` succeed but `int(...)` raise OverflowError (a subclass of ArithmeticError, NOT ValueError). Verified: `int(float('1e400'))` -> OverflowError, uncaught. This escapes the row loop, propagates out of _ingest_remote_segments -> out of _maybe_dispatch_remote (it is AFTER the try/except that wraps run_infer), and surfaces as a generic uncaught job failure rather than the clean `_failed(...)`/skip-the-row contract the rest of the function is built around. Worse, it aborts after some rows were already inserted via add_play_segment, leaving a partial set of cloud rows that _finalize_reingest never gets to reconcile (the old rows are not pruned, the new partial rows are not pruned). The docstring promises 'A malformed row is skipped, never aborts the ingest' ? this input violates that promise.
Fix: Catch (ValueError, OverflowError) (or `ArithmeticError`) around the `int(float(sc))` coercion, falling back to storing the raw string just like the existing ValueError branch. Same hardening is warranted for the start/end coercion if non-finite values should be rejected (see separate finding).

2. assistant (2026-06-22T15:03:12.043Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repository.

3. user (2026-06-22T15:03:13.212Z)

```
 backend/main.py                  | 100 ++++++++++++++++++
 tests/test_api_cloud_dispatch.py | 212 ++++++++++++++++++++++++++++++++++++++++
 2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:03:15.165Z)

Let me look at the actual diff in the cited region of `backend/main.py`.

5. user (2026-06-22T15:03:16.652Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
```

```
+++ b/backend/main.py
@@ -1,10 +1,12 @@
 import os
 import sys
 import argparse
+import csv
 import json
 import logging
 import shutil
 import socket
+import tempfile
 import threading
 import uuid
 import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
     return FileResponse(path, media_type=media, filename=name)


+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
(cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                    "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                           play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
+    == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
```

```
+
+    compute = get_compute_backend(global_config)
+    if compute is None:
+        return None  # default-OFF ? run the in-process segmenter below (unchanged)
+
+    storage_cfg = getattr(global_config, "storage", None)
+
+    def _failed(msg: str) -> Dict[str, Any]:
+        # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+        return {"video_id": video_id, "status": "failed", "message": msg,
+                "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+    # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+    try:
+        input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+    except ValueError as e:
+        return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+    if input_ref.backend == "local":
+        return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+                       "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+    if not out_root:
+        return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+                       "? that's where the Cloud Run job writes the result.")
+
+    notify("dispatching (cloud_run)")
+    logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+    spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+    try:
+        result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
+    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+        return _failed(f"cloud dispatch error: {e}")
+    if result.returncode != 0:
+        return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+    notify("ingesting (cloud_run)")
+    storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+    _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+    segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+    _finalize_reingest(video_id, segments, play_wm, cand_wm)
+    return {"video_id": video_id, "status": "success",
+            "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+            "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
```

```
         # if the run yields segments; otherwise drop the new partial rows and keep the old.
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+        # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+        # they're dropped on the cloud path in this slice ? a documented follow-up.)
+        remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+        if remote is not None:
+            segmentation_ran = True
+            response = remote
+            return response
         segmenter = segmenter_cls(db_instance, global_config)
         try:
             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
```

6. assistant (2026-06-22T15:03:18.951Z)

The finding cites lines 462-465. Let me read the exact code with line numbers to confirm the structure and the wrapping try/except.

7. user (2026-06-22T15:03:20.050Z)

```
441     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) ->
int:
442         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
443         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
444         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
445         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
446         backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
447         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
448         ref = storage.parse_uri(uri)
449         dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or
"intervals.csv")
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n
469
```

```

    def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
                               play_wm: int, cand_wm: int, notify) -> Optional[Dict[str,
Any]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
        fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend``
returns ``None``
        for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        compute = get_compute_backend(global_config)
        if compute is None:
            return None  # default-OFF ? run the in-process segmenter below (unchanged)

        storage_cfg = getattr(global_config, "storage", None)

        def _failed(msg: str) -> Dict[str, Any]:
            # Shape-match the in-process segmenter-fail branch + restore prior good rows
(none written yet).
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            return {"video_id": video_id, "status": "failed", "message": msg,
                    "results": [r.model_dump() for r in val_results], "play_segments": []}

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
        if input_ref.backend == "local":
            return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
                           "can't be read by the Cloud Run job. Put the video in your bucket
first.")
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
                           "? that's where the Cloud Run job writes the result.")

        notify("dispatching (cloud_run)")
        logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
        spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
        try:
            result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}")
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).")

        notify("ingesting (cloud_run)")
        storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
```

```
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
570                     "video_id": video_id,
```

8. assistant (2026-06-22T15:03:22.860Z)

Now let me verify the claim about `OverflowError` not being caught by `except ValueError`. Let me confirm the Python behavior.

9. user (2026-06-22T15:03:26.279Z)

```
'1e400' float= inf int -> RAISES OverflowError | is ValueError? False | is ArithmeticError? True
'inf' float= inf int -> RAISES OverflowError | is ValueError? False | is ArithmeticError? True
'-inf' float= -inf int -> RAISES OverflowError | is ValueError? False | is ArithmeticError? True
'nan' float= nan int -> RAISES ValueError | is ValueError? True | is ArithmeticError? False
'Infinity' float= inf int -> RAISES OverflowError | is ValueError? False | is ArithmeticError?
True
```

10. assistant (2026-06-22T15:03:31.408Z)

Confirmed: `int(float('1e400'))` (and `'inf'`, `'-inf'`, `'Infinity'`) raises `OverflowError`, which is NOT a subclass of `ValueError`, so the `except ValueError` on line 464 does NOT catch it. Note `'nan'` raises `ValueError` (caught), so the finding's specific examples are accurate.

Now let me verify the propagation claim ? that this escapes `_ingest_remote_segments`, propagates out of `_maybe_dispatch_remote` (it's AFTER the try/except wrapping `run_infer`), and surfaces as an uncaught failure. Looking at the code: `_ingest_remote_segments` is called at line 520, which is OUTSIDE/after the try/except at 510-514. So an exception there propagates up. Let me check the outer caller `_execute_processing` and the job runner to confirm what happens.

11. user (2026-06-22T15:03:32.069Z)

```
595             logger.info(f"[API] Using segmenter: {seg_name}")
596             notify(f"segmenting ({seg_name})")
597             segmenter_actual = seg_name
598             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605             # they're dropped on the cloud path in this slice ? a documented follow-up.)
606             remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607             if remote is not None:
608                 segmentation_ran = True
609                 response = remote
610                 return response
611             segmenter = segmenter_cls(db_instance, global_config)
612             try:
613                 result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614             except Exception:
615                 # A raising segmenter must not leave half-written rows: restore the pre-run
616                 # state (same destroy-after-confirm contract as the Failure branch), then
let
617                 # the job worker record the failure.
618                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619                 raise
620             segmentation_ran = True
621
622             if isinstance(result, Failure):
623                 logger.error(f"[API] Segmenter failed: {result.message}")
```

```
624                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625                 response = {
626                     "video_id": video_id,
627                     "status": "failed",
628                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
629                     "results": [r.model_dump() for r in val_results],
630                     "play_segments": [],
631                 }
632                 return response
633 
634             segments = result.value if hasattr(result, "value") else result
635             notify("finalizing")
636             _finalize_reingest(video_id, segments, play_wm, cand_wm)
637 
638             response = {
639                 "video_id": video_id,
640                 "status": "success",
641                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
642                 "results": [r.model_dump() for r in val_results],
643                 "play_segments": segments,
644             }
645             return response
646         finally:
647             # Always emit the per-video observability report (next to the video, or to
648             # report.output_dir). Never raises. Surface the paths in the response.
649             paths = emit_indexer_report(
650                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
651                 val_results=val_results, val_context=val_context,
652                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
653                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
654             )
655             if paths and isinstance(response, dict):
656                 response["reports"] = paths
657 
658 
659     @app.post("/api/process", status_code=202)
660     def process_video(req: ProcessRequest, request: Request):
661         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
662 
663         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
664         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
665         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
666         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
667         """
668         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
669         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
670         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
671         try:
672             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
673         except edge_auth.EdgeAuthError as e:
674             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
```

12. assistant (2026-06-22T15:03:38.757Z)

Now I've confirmed the propagation path. The key observations:

1. Line 520 (`_ingest_remote_segments`) is called OUTSIDE the try/except at 510-514 (which only wraps `run_infer`). So an `OverflowError` from `_ingest_remote_segments` propagates up through `_maybe_dispatch_remote` un-caught.

2. In `_execute_processing`, the call at line 606 (`_maybe_dispatch_remote`) is inside the outer `try:` (line 564) but the only `except` is the segmenter-specific one at line 614 (which wraps `segmenter.process_video`, a different statement). There's no `except` covering line 606 ? only a `finally`. So the exception propagates out of `_execute_processing`.

Let me verify the partial-row concern: that `add_play_segment` writes rows before the OverflowError aborts, and that nothing prunes them. Let me check the order in `_ingest_remote_segments` and whether the caller's `_failed` pruning is reachable.

13. user (2026-06-22T15:03:41.115Z)

```
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:285:def _finalize_reingest(video_id: str,
segments, play_wm: int, cand_wm: int) -> None:
C:\Users\avidu\Projects\bhi-cloudrun\backend\infrastructure\database.py:438:    def
prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
C:\Users\avidu\Projects\bhi-cloudrun\backend\infrastructure\database.py:459:    def
add_play_segment(self, video_id: str, start_time: float, end_time: float, server: Optional[str]
= None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None) ->
Optional[int]:
```

14. assistant (2026-06-22T15:03:45.875Z)

I notice `prune_segments` in database.py at line 438 has signature `(video_id, *, play_upto=None, cand_upto=None)` but the calls in main.py use `play_after=` / `cand_after=`. Let me check this ? but that's a separate concern. Let me verify the prune signature used in `_failed` and `_finalize_reingest`.

15. user (2026-06-22T15:03:46.255Z)

```
438         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439                            play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
440                            cand_after: Optional[int] = None) -> None:
441             """Delete play/candidate segments by id range (events cascade from
play_segments).
442
443             Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444             successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
445             and keep the OLD ones when the run failed/produced nothing (``*_after``).
446             """
447             with self._connect() as conn:
448                 cur = conn.cursor()
449                 if play_upto is not None:
450                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451                 if play_after is not None:
452                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453                 if cand_upto is not None:
454                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455                 if cand_after is not None:
456                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457                 conn.commit()
458
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
```

16. assistant (2026-06-22T15:03:51.231Z)

Good, `prune_segments` supports both `play_after`/`cand_after`. So the `_failed` path correctly prunes new partial rows. But the critical point is: when `OverflowError` is raised from `_ingest_remote_segments` (line 520), it propagates BEFORE reaching `_finalize_reingest` (523) and the `_failed` path is never invoked (we're past all the `_failed` returns) ? so the partial rows already written by `add_play_segment` (line 466, for rows processed before the bad row) are NEVER pruned. This confirms the partial-state corruption claim.

Now let me verify the docstring promise and check whether the tests cover this case.

17. user (2026-06-22T15:03:52.251Z)

```
22:from backend.infrastructure.database import Database
23:from backend.infrastructure.execution_policy import ExecutionPolicy
60:                w.writerow(["start", "end", "shot_count", "ending_reason"])
118:    assert {md.get("shot_count") for md in metas} == {5, 3}
```

18. user (2026-06-22T15:03:55.037Z)

```
50              out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
51              done_uri = argv[argv.index("--done-uri") + 1]
52              if self.returncode == 0:
53                  outputs = os.path.join(out_uri, "outputs", self.video_id)
54                  os.makedirs(outputs, exist_ok=True)
55                  with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
56                      json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
57                                 "status": "success"}, f)
58                  with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
59                      w = csv.writer(f)
60                      w.writerow(["start", "end", "shot_count", "ending_reason"])
61                      for s, e, sc, er in self.rows:
62                          w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
63              os.makedirs(os.path.dirname(done_uri), exist_ok=True)
64              with open(done_uri, "w", encoding="utf-8") as f:
65                  json.dump({"video_id": self.video_id, "returncode": self.returncode,
66                             "segment_count": len(self.rows), "status": "success"}, f)
67              return "exec-1"
68
69
70      @pytest.fixture
71      def cloud_env(tmp_path, monkeypatch):
72          """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
73          identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
74          db = Database(str(tmp_path / "t.db"))
75          bucket = tmp_path / "bucket"
76          bucket.mkdir()
77          cfg = AppConfig.model_validate({
78              "deployment": {"compute_target": "cloud_run"},
79              "storage": {"backend": "local", "output_root": str(bucket)},
80          })
81          monkeypatch.setattr(m, "db_instance", db)
82          monkeypatch.setattr(m, "global_config", cfg)
83          monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
84          # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
85          fake_local = tmp_path / "fetched.mp4"
86          fake_local.write_bytes(b"FAKEVIDEOBYTES")
87          monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
str(fake_local))
88          monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
```

```
89          return db, bucket, monkeypatch
90
91
92      def _inject_fake(monkeypatch, fake):
93          """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
94          real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token."""
95          real = compute_pkg.get_compute_backend
96          monkeypatch.setattr(compute_pkg, "get_compute_backend",
97                              lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))
98
99
100     def _meta(row):
101         md = row.get("metadata")
102         return json.loads(md) if isinstance(md, str) else (md or {})
103
104
105     def test_cloud_dispatch_ingests_segments(cloud_env):
106         db, _bucket, monkeypatch = cloud_env
107         fake = _FakeCloudClient()
108         _inject_fake(monkeypatch, fake)
109
110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"
113         assert len(out["play_segments"]) == 2
114         rows = db.query_play_segments(out["video_id"], {})
115         assert len(rows) == 2
116         metas = [_meta(r) for r in rows]
117         assert all(md["source"] == "cloud_run" for md in metas)
118         assert {md.get("shot_count") for md in metas} == {5, 3}
119         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
120
121
122     def test_cloud_dispatch_builds_correct_spec(cloud_env):
123         _db, bucket, monkeypatch = cloud_env
124         fake = _FakeCloudClient()
```

19. assistant (2026-06-22T15:03:58.733Z)

The tests cover valid `shot_count` values (5, 3) and likely malformed-string cases, but let me confirm there's no test for the non-finite `inf`/`1e400` input. Let me check the full test for malformed rows.

20. user (2026-06-22T15:04:00.004Z)

```
105:def test_cloud_dispatch_ingests_segments(cloud_env):
122:def test_cloud_dispatch_builds_correct_spec(cloud_env):
139:def test_cloud_failure_marks_failed_no_rows(cloud_env):
153:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
165:def test_default_off_runs_in_process(tmp_path, monkeypatch):
191:def test_full_api_process_to_query_via_cloud(cloud_env):
```

21. assistant (2026-06-22T15:04:03.253Z)

No test exercises a non-finite `shot_count` (no `inf`/`1e400` test). The finding is not contradicted by test coverage.

Let me do one final confirmation by directly reproducing the failure path through the actual code, to be sure the `OverflowError` truly escapes (and isn't somehow caught upstream). I'll write a tiny isolated reproduction against the real function logic.

22. user (2026-06-22T15:04:07.103Z)

```
valid 5: {'shot_count': 5}
junk abc: {'shot_count': 'abc'}
nan: {'shot_count': 'nan'}
inf -> UNCAUGHT OverflowError
1e400 -> UNCAUGHT OverflowError
```

23. assistant (2026-06-22T15:04:21.214Z)

The reproduction confirms every claim. Let me assess severity honestly before finalizing.

The finding holds up technically and completely. Now weighing severity:

Confirmed facts:
- `int(float('inf'))` / `int(float('1e400'))` raises `OverflowError`, which is NOT a `ValueError` subclass, so the `except ValueError` at line 464 does not catch it. (`'nan'` raises `ValueError` and IS caught ? so the finding correctly picked the right examples.)
- Line 520 (`_ingest_remote_segments`) is outside the try/except at 510-514 (which only wraps `run_infer`), so the exception escapes `_maybe_dispatch_remote`.
- In `_execute_processing`, the call at line 606 is covered only by a `finally` (no matching `except` for this statement ? the `except` at 614 wraps the different `segmenter.process_video` call), so the exception propagates out as a generic job failure.
- Partial rows: `add_play_segment` (line 466) commits rows for all good rows preceding the bad one; the `OverflowError` aborts before `_finalize_reingest` (line 523) and bypasses the `_failed` pruning, leaving partial new rows + un-pruned old rows. This violates the docstring's "A malformed row is skipped, never aborts the ingest" promise.

Severity weighting (downgrade considerations):
- The whole `cloud_run` path is DEFAULT-OFF (gated behind `deployment.compute_target == "cloud_run"`).
- The `shot_count` source is `intervals.csv` written by the project's own Cloud Run worker, not arbitrary external/user input ? so a non-finite `shot_count` is unlikely in practice unless the worker emits it (a `shot_count` realistically can't be infinite). This is a robustness/defense-in-depth gap, not an externally-triggerable exploit on a hot path.

The finding labels it `major`. It is a real defect with a real partial-state corruption consequence, but it requires the cloud path enabled AND the trusted worker emitting a non-finite `shot_count` (implausible for a shot count). The blast radius is real but the trigger likelihood is low given the trusted, structurally-bounded source. The fix is trivial and correct (catch `(ValueError, OverflowError)`). I'd land it as a real defect at **minor** severity rather than major ? the inconsistent exception handling (the sibling `start`/`end` block also only catches `ValueError`, not `OverflowError`) is a genuine correctness gap worth fixing, but the practical exploitability is low.

24. user (2026-06-22T15:04:29.264Z)

Structured output provided successfully

25. assistant (2026-06-22T15:04:37.177Z)

Verified the finding against the cited code. It holds up as a real defect, downgraded from major to minor.

Key file: `C:/Users/avidu/Projects/bhi-cloudrun/backend/main.py:462-465`

Confirmed:
- `int(float('inf'/'1e400'/'-inf'/'Infinity'))` raises `OverflowError`, not caught by the `except ValueError` (reproduced directly; `'nan'` correctly raises `ValueError` and is caught).
- The exception escapes `_ingest_remote_segments` (called at line 520, outside the `try/except` at 510-514 that wraps only `run_infer`), escapes `_maybe_dispatch_remote`, and propagates out of `_execute_processing` (the call at line 606 is covered only by `finally`).
- Partial-state hazard is real: rows committed via `add_play_segment` (line 466) before the bad row are never pruned, and `_finalize_reingest` never runs ? violating the docstring's "A

malformed row is skipped, never aborts the ingest."

Downgrade rationale: the whole `cloud_run` path is default-OFF, and `shot_count` comes from the project's own trusted Cloud Run worker `intervals.csv`, where a non-finite shot count is implausible ? so it is a genuine robustness/consistency gap and partial-write hazard, but low real-world trigger likelihood. The proposed fix (`except (ValueError, OverflowError)`, with the same hardening for the start/end coercion at lines 455-457) is correct.